MyRCA: Unsupervised Root Cause Analysis in Microservices via Distributed Tracing, Causal Scoring, and MLOps Integration

Hung Phung-Quang
Faculty of Computer Networks and Communications
University of Information Technology VNU-HCM
Ho Chi Minh City, Vietnam
22520524@gm.uit.edu.vn

Hung Tran-Tien
Faculty of Computer Networks and Communications
University of Information Technology VNU-HCM
Ho Chi Minh City, Vietnam
22520527@gm.uit.edu.vn

Tuan Le-Anh
Faculty of Computer Networks and Communications
University of Information Technology VNU-HCM
Ho Chi Minh City, Vietnam
tuanla@uit.edu.vn

***Abstract*—Microservice architectures introduce significant operational complexity because failures propagate across distributed service dependencies, making root cause analysis (RCA) difficult with traditional log- and metric-based methods. This paper presents MyRCA, an unsupervised two-step RCA framework for microservices built on distributed tracing data. In the first step, a Transformer Autoencoder detects anomalous spans through reconstruction error, achieving F1-score of 0.8772 on the public TraceRCA dataset and 0.8392 on the self-collected MeetingTeam dataset, with the highest Recall among six evaluated models. In the second step, the MyRCA algorithm ranks suspect services by combining a Causal Score and a Jaccard Score with a Knowledge Base that captures synchronous and asynchronous API relationships inferred from historical traces. On the TraceRCA benchmark, MyRCA achieves A@1 = 0.909, outperforming the TraceRCA baseline (0.83) on all ranking metrics while remaining fully unsupervised. An integrated MLOps pipeline with Airflow, MLflow, and Evidently automates drift-triggered retraining. End-to-end RCA latency stays below 13.1 s at 784 spans/s, meeting the 30-second SLO across all tested load scenarios.**

***Index Terms*—microservices, root cause analysis, distributed tracing, anomaly detection, transformer autoencoder, MLOps.**

I. Introduction

Microservice architecture has become the dominant design pattern for cloud-native systems because it enables independent deployment, fine-grained scaling, and technology flexibility [1]. However, these properties also create significant operational challenges. When a single service experiences a fault, the error propagates through synchronous call chains and appears as degraded latency or elevated error rates in multiple downstream services, making it difficult to distinguish the root cause from the symptoms.

Distributed tracing addresses this gap by recording the full execution path of each request as it traverses the system [2]. Each trace carries timing, status, and structural context for every service invocation (span), providing the causal evidence needed for RCA. Recent work has exploited trace data for RCA [3]–[5], but key limitations remain. First, most approaches treat anomaly detection and RCA as separate problems, without exploiting anomaly signals as a directed pre-filter for RCA. Second, existing methods typically do not distinguish between synchronous and asynchronous API relationships, which changes how faults propagate. Third, machine learning models used for anomaly detection degrade over time as service behavior evolves (data drift), yet few systems integrate automated monitoring and retraining.

A. Related Work

Trace-based RCA methods can be grouped into three families. Statistical approaches such as TraceRCA [3] rank services by how frequently they appear in anomalous traces; the method does not model causal propagation structure. Causal discovery approaches such as Microscope [5] construct a causal graph between services using the PC algorithm, but scale poorly to large service graphs. Graph-based propagation approaches such as MicroRCA [4] combine latency analysis with a service dependency graph and a random-walk scorer. Supervised fault-localization methods such as MEPFL [6] achieve high Top-1 accuracy by training on labeled fault-injection data, but this labeling requirement limits applicability in environments where fault data is scarce.

For anomaly detection on trace data, deep learning autoencoders (LSTM AE [10], GRU AE [11], Transformer AE [9]) reconstruct normal trace patterns and flag spans whose reconstruction error exceeds a learned threshold. However, these models are rarely integrated with downstream RCA logic or with an automated retraining pipeline.

B. Contributions

This paper makes the following contributions:

- A Transformer Autoencoder with a bi-stream encoder that fuses latency time-series with six span context features for span-level anomaly detection across multiple microservice deployments.

- The MyRCA algorithm, which builds a Knowledge Base of sync/async API relationships from historical traces and ranks suspect services by combining a Causal Score (measuring fault propagation depth) and a Jaccard Score (measuring presence in anomalous traces).
- An end-to-end system on Kubernetes with an MLOps pipeline (Airflow, MLflow, Evidently) that monitors data drift and triggers retraining automatically, validated on two datasets with performance benchmarks.

The rest of this paper is organized as follows. Section II describes the system architecture. Section III describes the Trace-RCA Dashboard. Section IV presents the anomaly detection module. Section V details the MyRCA algorithm. Section VI describes the MLOps pipeline. Section VII reports experimental evaluation. Section VIII concludes.

II. System Architecture

The MyRCA system is deployed on Kubernetes and organized into five functional layers, as illustrated in Fig. 1.

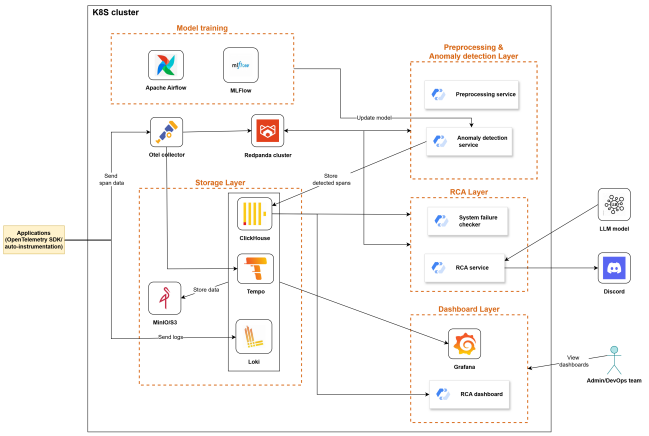


Fig. 1. MyRCA system architecture showing five functional layers from data collection through the Trace-RCA Dashboard.

Data Collection Layer. OpenTelemetry (OTel) SDK and auto-instrumentation are embedded in each microservice to emit spans and logs. An OTel Collector [13] receives telemetry over gRPC/HTTP and distributes it in parallel to Grafana Tempo (trace storage) and Redpanda (Kafka-compatible streaming).

Storage Layer. Grafana Tempo stores raw traces and supports TraceID-based lookup. Grafana Loki aggregates application logs. ClickHouse stores preprocessed spans using a MergeTree engine with `ORDER BY (service_name, timestamp)` and daily partitioning for fast RCA queries. A tiered storage policy moves data older than one day to AWS S3, maintaining query access while controlling cost.

Message Streaming Layer. Redpanda carries raw spans from the OTel Collector to the Preprocessing Service, preprocessed spans to the Anomaly Detection Service, and failure signals from the System Failure Checker to the RCA Service.

Processing Layer. The Preprocessing Service normalizes raw spans. The Anomaly Detection Service runs the Transformer Autoencoder model. The System Failure Checker monitors anomaly ratios in one-minute windows and fires a Discord alert and an RCA trigger when more than 50% of spans in any API group are anomalous. The RCA Service executes the MyRCA algorithm and stores incident records in PostgreSQL.

Dashboard Layer. Grafana provides metric and trace visualization. The Trace-RCA Dashboard is a standalone React/FastAPI web application that lists incidents, shows RCA results, and exposes admin controls for user roles and API keys.

III. Trace-RCA Dashboard

The dashboard is organized into six modules covering 19 user-facing functions (Fig. 2): Authentication, Incident Monitoring, Application Filter, External Observability Links, Navigation, and Admin Configuration.

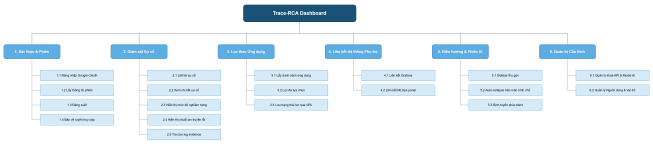


Fig. 2. Trace-RCA Dashboard: six functional modules and 19 user-facing functions.

Authentication uses Google OAuth 2.0 with JWT sessions and three access roles: Admin, Editor, and Viewer. **Incident Monitoring** lists all detected incidents with filters for application, service, severity (AUTO / WARNING / ESCALATE), and time window. The incident detail view (Fig. 3) presents the full two-stage RCA result: Stage 1 structural rankings via $CS \times JS$ score bars, LLM-identified root cause with confidence score, propagation path, and per-service log evidence annotated as INTRINSIC or PROPAGATED. **Admin Configuration** lets admins manage LLM API keys across three providers (Anthropic, OpenAI, Google AI) — key activation is atomic to prevent split-brain — and perform CRUD operations on user accounts and role assignments.

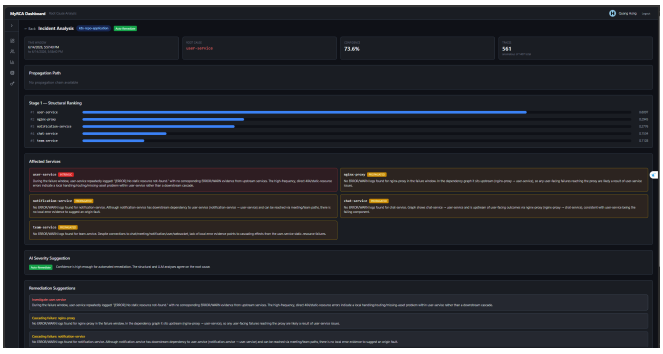


Fig. 3. Incident detail view showing Stage 1 structural ranking, LLM root cause analysis, and per-service log evidence classified as INTRINSIC or PROPAGATED.

Incident data is stored in a denormalized PostgreSQL table with `JSONB` columns for `stage1_ranking` and `log_evidence`, indexed on `(created_at DESC, app_id)` for fast listing and per-application filtering.

IV. Anomaly Detection with Transformer Autoencoder

A. Preprocessing and Feature Construction

Each span is represented as a sequence of latency values grouped by API key (`app_id, service, operation, http_status`). A sliding window of fixed length is applied to each group to produce input sequences for the model. Six context features are extracted per span: `app_id`, `service`, `operation`, `parent_service`, `parent_operation`, and `http_status`. HTTP status codes are bucketed into groups (2xx, 3xx, 4xx, 5xx); spans without HTTP use the value 0. Latency is standardized with a per-group StandardScaler.

B. Model Architecture

The Transformer Autoencoder receives two parallel inputs: a latency time-series and the embedded context features. Context embeddings are produced by learned categorical embedding tables (service dimension 4, operation dimension 6) and concatenated with the latency vector.

Encoder. The combined input is projected to 64 dimensions and augmented with positional encoding. Two Transformer Encoder layers (4 attention heads each) learn temporal dependencies. Mean pooling over the time dimension compresses the output to a 32-dimensional latent vector.

Decoder. The latent vector is expanded along the time dimension, concatenated with the original context embeddings, and passed through two additional Transformer Encoder layers to reconstruct the input latency sequence.

Anomaly threshold. After training on normal traces, the reconstruction error is computed on a held-out evaluation set assumed to contain mainly normal spans. The 99th percentile (p99) of this error distribution is used as the anomaly threshold, so that at most 1% of normal spans are flagged. The threshold is saved together with the model artifact.

V. MyRCA Algorithm

A. Knowledge Base Construction

The Knowledge Base encodes whether each caller–callee API pair communicates synchronously or asynchronously, since fault propagation follows synchronous (blocking) calls but not asynchronous (fire-and-forget) ones. For each parent–child span pair, the OTel `kind` field is inspected first: `CLIENT`→`SERVER` pairs are synchronous; `PRODUCER`→`CONSUMER` pairs are asynchronous.

For internal spans without these kinds, the *Latency Sensitivity Index* (LSI) is computed from historical trace data. LSI compares the mean latency of the parent span when the child processes quickly (below median) versus slowly (above p90). If the parent is consistently slower when the child is slow (LSI > 1.0) and the validity ratio exceeds 0.95, the pair is classified as synchronous; otherwise asynchronous. The resulting mapping from (parent API, child API) to synchrony label is persisted as the Knowledge Base.

B. Causal Score and Jaccard Score

Given a window of traces containing anomalous spans, MyRCA runs in four steps.

Step 1 (Graph construction). Each trace is organized as a directed acyclic graph (DAG) where nodes are spans annotated with their anomaly flag and edges represent caller–callee relationships.

Step 2 (Causal Score). For each anomalous span, a depth-first search traverses the synchronous downstream subgraph and counts the number of anomalous descendant spans. The Causal Score reflects the principle that a service with few downstream anomalies is more likely to be the root cause than one whose anomalous status is explained by propagated faults:

$$CS(s) = \frac{1}{1 + w_d \cdot N_{\text{down}}(s)} \tag{1}$$

where $N_{\text{down}}(s)$ is the number of anomalous downstream synchronous spans reachable from s, and w_d is the downstream penalty weight (default 2). Scores are aggregated per service by taking the mean across all traces.

Step 3 (Jaccard Score). The Jaccard Score measures the fraction of traces containing service s that are anomalous:

$$JS(s) = \frac{|T_{\text{abn}} \cap T_s|}{|T_{\text{abn}} \cup T_s|} \tag{2}$$

where T_{abn} is the set of anomalous traces and T_s is the set of all traces that pass through service s. A low Jaccard Score indicates the service appears in many normal traces, penalizing ubiquitous services that are unlikely to be root causes.

Step 4 (Ranking). The final score is the product of both scores:

$$\text{score}(s) = CS(s) \times JS(s) \tag{3}$$

Services are ranked in descending order and returned as the top-k RCA output.

VI. MLOps Pipeline

The MLOps pipeline (Fig. 4) is implemented as an Apache Airflow [15] DAG and can be triggered either automatically on a schedule or manually by an administrator.

Drift Detection. The pipeline starts by comparing the current span dataset against the training dataset. Evidently AI computes the Population Stability Index (PSI) on the `duration_ns` distribution. Drift is declared if at least 50% of monitored fields show a significant distributional shift (PSI > 0.2) or if new API endpoints not present during training are detected. If no drift is detected, the pipeline skips retraining.

Preprocessing and Split. Spans are loaded from ClickHouse, encoded with the same LabelEncoder and StandardScaler used at inference, and split into training and validation sets.

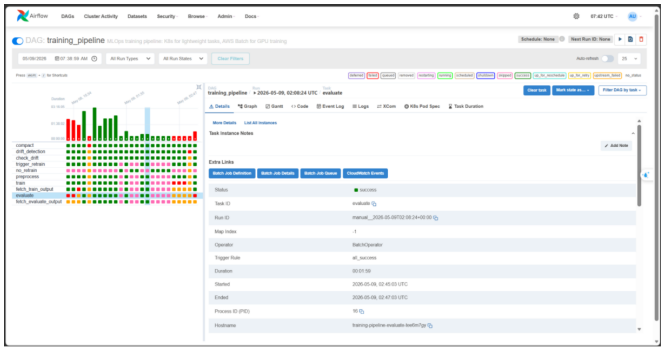


Fig. 4. Apache Airflow DAG for the MLOps training pipeline, showing drift detection branching, preprocessing, training, validation, and model deployment steps.

Training. The Transformer Autoencoder is retrained on the updated dataset. Light preprocessing tasks run as KubernetesPodOperator jobs; GPU-intensive training jobs run on AWS Batch Spot instances (g4dn.xlarge with T4 GPU) to control cost. Typical retraining duration is 15–35 seconds.

Validation and Deployment. The retrained model is evaluated on the validation set; the p99 reconstruction error sets the new anomaly threshold. Experiment metadata, model artifacts, and the threshold are registered in MLflow Model Registry [14]. When the new model's F1-score exceeds the current production model, MLflow automatically promotes it to the "champion" alias and the inference service hot-reloads the artifact from S3.

VII. Experimental Evaluation

A. Datasets

TraceRCA. We use Dataset A from [3], collected from the 41-service TrainTicket benchmark on a 7-node Kubernetes cluster. It contains 176 fault scenarios (58 application-bug, 59 CPU-exhaustion, 59 network-delay) across 242,259 traces, of which 9.36% are anomalous.

MeetingTeam. We collect a self-labeled dataset from a 7-service Chat-Web application deployed on 5 AWS EC2 `t3.medium` instances. K6 load scripts simulate 21 API call types. Faults are injected using `tc netem` (90 ms delay, 10% packet loss) and service pod removal. The dataset contains 23 fault scenarios (14 latency, 9 application faults) with 250,000 traces, of which 32.3% are anomalous.

B. Anomaly Detection Results

Table I and Table II report anomaly detection performance. All autoencoder models are trained on normal traces only (unsupervised). Thresholds are selected to maximize F1-score on the evaluation set.

GRU AE achieves the highest F1 on TraceRCA (0.8930), but its cross-dataset gap is 0.063 points. Transformer AE achieves the best F1 on MeetingTeam (0.8392) and the highest Recall on both datasets (0.9284 and 0.8965), with a cross-dataset gap of only 0.038 points, indicating better generalization. In anomaly detection for monitoring, high Recall is preferable because missed anomalies (false negatives) are more costly than false alarms. Transformer AE is therefore selected as the production model.

TABLE I
Anomaly Detection on TraceRCA Dataset

Model	F1	Recall	Precision
LSTM AE [10]	0.8811	0.9197	0.8457
GRU AE [11]	0.8930	0.9217	0.8660
TCN AE	0.8262	0.8348	0.8178
Transformer AE [9]	**0.8772**	**0.9284**	0.8312
Z-score	0.6633	0.9627	0.5060
Isolation Forest [12]	0.6849	0.6890	0.6809

TABLE II
Anomaly Detection on MeetingTeam Dataset

Model	F1	Recall	Precision
LSTM AE	0.8205	0.8250	0.8158
GRU AE	0.8295	0.8735	0.7896
TCN AE	0.8184	0.8589	0.7817
Transformer AE	**0.8392**	**0.8965**	0.7887
Z-score	0.3394	0.2044	1.0000
Isolation Forest	0.4175	0.9228	0.2698

C. RCA Comparison with Baselines

Table III compares MyRCA against five baselines on the TraceRCA dataset. Baseline figures are taken directly from the respective papers. Note that MEPFL [6] is a supervised method trained on labeled fault-injection data, while all other methods including MyRCA are unsupervised.

MyRCA achieves A@1 = 0.909, improving over TraceRCA by 0.079 points and over all other unsupervised baselines by large margins (RCSF: +0.389, Microscope: +0.349, TraceAnomaly: +0.419). On ranking metrics, MyRCA outperforms the supervised MEPFL (MAR 1.097 vs. 1.420; MFR 1.119 vs. 1.380) despite not requiring any fault-labeled training data. Microscope and TraceAnomaly exhibit high MAR values (3.64 and 4.42), indicating that on average 3–4 candidates must be inspected before reaching the true root cause.

Table IV shows MyRCA results on the MeetingTeam dataset. A@2 = A@3 = 1.0 indicates that in every test scenario the true root cause appears within the top-2 recommendations, and MAR = MFR = 1.08 is near the ideal value of 1.0.

D. End-to-End Performance

Performance was measured on a Kubernetes cluster of five `t3.medium` AWS EC2 nodes at four span-rate scenarios (Rate 1–4: 358, 513, 627, 784 spans/s) over 5-minute runs. SLOs are defined following the Google SRE framework [16]: ML inference ≤ 20 ms/span, E2E anomaly < 30 s at ≤ 500 spans/s, RCA algorithm < 10 s/run, E2E RCA < 30 s/run.

As shown in Table V, all RCA SLOs are met across all four load levels (E2E RCA stays 7.7–13.1 s, well below 30 s). Compared to manual microservice fault diagnosis, which typically takes tens of minutes [17], MyRCA achieves 137–234×

TABLE III
RCA COMPARISON ON TRACERCA DATASET

Algorithm	A@1	A@3	MAR	MFR	Type
MyRCA (ours)	**0.909**	**0.994**	**1.097**	**1.119**	Unsup.
TraceRCA [3]	0.830	0.970	1.390	1.340	Unsup.
MEPFL [6]	0.940	0.970	1.420	1.380	Sup.
RCSF [7]	0.520	0.930	1.680	1.600	Unsup.
Microscope [5]	0.560	0.700	3.640	3.470	Unsup.
TraceAnomaly [8]	0.490	0.630	4.420	4.340	Unsup.

TABLE IV
MYRCA RESULTS ON MEETINGTEAM DATASET

Algorithm	A@1	A@2	A@3	MAR
MyRCA (ours)	0.920	1.000	1.000	1.080

speedup. Anomaly detection SLOs are met at $\leq$500 spans/s; at higher rates the E2E anomaly latency increases non-linearly (7.6$\times$ when span rate increases from 358 to 784 spans/s), which we attribute to Redpanda partition imbalance under high throughput. RCA algorithm execution time scales linearly (4.5–9.1 s/run), consistent with ClickHouse's indexed query performance. Resource usage stays within SLO bounds at all rates (CPU 1,155–2,132 millicores; memory 7.7–8.1 GiB).

The estimated operational cost on AWS us-east-1 is $188.58/month for infrastructure (five `t3.medium` EC2 instances, 200 GB EBS, 700 GB S3).

VIII. CONCLUSION

This paper presented MyRCA, a two-step unsupervised framework for root cause analysis in microservices. The Transformer Autoencoder achieves the best cross-dataset generalization for span-level anomaly detection with a 0.038-point F1 gap across two datasets and the highest Recall among six evaluated models. The MyRCA scoring algorithm improves over the TraceRCA baseline by 0.079 A@1 points on the TraceRCA benchmark and outperforms the supervised MEPFL on all ranking metrics while requiring no labeled fault data. An MLOps pipeline with PSI-based drift detection and automatic retraining ensures that the anomaly detection model adapts to behavioral changes in the monitored system. End-to-end RCA completes within 13.1 s at 784 spans/s, satisfying all defined SLOs.

This work has two main limitations. First, scalability degrades above 500 spans/s because the current deployment does not horizontally scale the anomaly detection pods; Redpanda partition imbalance further amplifies this bottleneck. Second, anomaly detection is univariate (latency only) and does not integrate resource metrics such as CPU and memory saturation.

Future directions include horizontal auto-scaling of the anomaly detection layer, multivariate anomaly detection combining latency with resource and error-rate signals, graph-based deep learning models for anomaly detection, and A/B testing of retrained models against production traffic before full promotion.

TABLE V
END-TO-END PERFORMANCE ACROSS LOAD SCENARIOS

Rate	Spans/s	ML infer. (ms)	E2E anom. (s)	E2E RCA (s)
1	358	9	5.1	7.7
2	513	11	10.0	8.6
3	627	12	17.6	10.5
4	784	16	38.6	13.1

REFERENCES

[1] N. Dragoni, S. Dustdar, S. T. Larsen, and M. Mazzara, "Microservices: Migration of a mission critical system," *arXiv preprint arXiv:1704.04173*, 2017.

[2] B. H. Sigelman *et al.*, "Dapper, a large-scale distributed systems tracing infrastructure," Google Technical Report dapper-2010-1, 2010.

[3] Z. Li *et al.*, "Practical root cause localization for microservice systems via trace analysis," in *Proc. IEEE/ACM 29th Int. Symp. Quality of Service (IWQoS)*, 2021, pp. 1–10, doi: 10.1109/IWQOS52092.2021.9521340.

[4] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2020, pp. 1–9, doi: 10.1109/NOMS47738.2020.9110353.

[5] J. Lin, P. Chen, and Z. Zheng, "Microscope: Pinpoint performance issues with causal graphs in micro-service environments," in *Int. Conf. Service-Oriented Computing (ICSOC)*, 2018, pp. 3–20, doi: 10.1007/978-3-030-03596-9_1.

[6] X. Zhou *et al.*, "Latent error prediction and fault localization for microservice applications by learning from system trace logs," in *Proc. 27th ACM Joint Meeting ESEC/FSE*, 2019, pp. 683–694, doi: 10.1145/3338906.3338961.

[7] K. Wang *et al.*, "A methodology for root-cause analysis in component based systems," in *Proc. IEEE/ACM 23rd Int. Symp. Quality of Service (IWQoS)*, 2015, pp. 243–248, doi: 10.1109/IWQoS.2015.7404741.

[8] P. Liu *et al.*, "Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks," in *Proc. IEEE 31st Int. Symp. Software Reliability Engineering (ISSRE)*, 2020, pp. 48–58, doi: 10.1109/ISSRE5003.2020.00014.

[9] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014, pp. 1724–1734, doi: 10.3115/v1/D14-1179.

[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 1, pp. 1–39, 2012.

[13] OpenTelemetry Authors, "OpenTelemetry documentation," 2024. [Online]. Available: https://opentelemetry.io/docs/.

[14] M. Zaharia *et al.*, "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

[15] Apache Software Foundation, "Apache Airflow documentation," 2026. [Online]. Available: https://airflow.apache.org/docs/.

[16] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[17] Y. Gan *et al.*, "Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices," in *Proc. 24th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 19–33, doi: 10.1145/3297858.3304004.